Adding a release: the repeatable procedure

- Adding a release: the repeatable procedure
  - Contents
  - 1. Why revision 1 was withdrawn
  - 2. Names, paths and remotes
  - 3. Preconditions
  - 4. The procedure
  - 5. Rollback
  - 6. Tag message template
  - 7. What can go wrong
  - 8. Evidence to keep for each release

Adding a release: the repeatable procedure

Purpose	One repeatable way to cut a numbered development release of the platform: build once into a side directory, give the candidate a fingerprint, produce a verdict against that fingerprint, back up, promote without rebuilding, confirm what is served, tag from the remote's tag list, push
Constitution	11.4.230(D) (incremental delivery), 11.4.235 (build and deploy once source is proven correct), 11.4.236 (QA-deploy-readiness gate), 11.4.264 (build once, promote one immutable artifact), 11.4.113 (never force-push), 11.4.151 (project-prefixed tag names), 11.4.40 (full-suite retest before a release tag), 11.4.61 (metadata table and contents)
Revision	3 (2026-10-21). Revision 1 described a procedure that cannot work (see part 1) and is withdrawn. Revision 2 was reviewed NO-GO on documentation only; revision 3 adds what releases 0.0.3 and 0.0.4 taught (the pre-deploy safety boot, the verdict vocabulary and the shipping-over-a-verdict rule, the stamp, the hazard of a later plain `start.sh`, a step 6 that survives being pasted)
Status	Draft for independent review (11.4.142)

Basis	The artifacts that releases 0.0.2, 0.0.3 and 0.0.4 left in `workshop` (`evidence/release/N/`: `MANIFEST.json`, `verdict.json`, `BUILD_STAMP`, `logs/`; for 0.0.4 also `server.stamp` and `logs/predeploy-safety.txt`; `docs/releases/N.md`) and on disk (`$HOME/.cache/workshop-release/N/`: `bin`, `web`, `stamp.txt`, `bundle.hash`, `web.filehashes.txt`, `pre-deploy-backup`), plus the scripts as they are on `workshop` `main` at `827e79e` (`scripts/_common.sh`, `restart.sh`, `start.sh`, `stop.sh`, `build.sh`, `verify.sh`, `run-isolated.sh`). The command lines the operators used for the promotion and the pre-deploy safety boot were **not preserved in a log**: they are written here from the scripts' own contracts and are **UNCONFIRMED** as replays of 0.0.2, 0.0.3 or 0.0.4. Executed by the author of this revision and recorded under "Checked in revision 3" in part 4: the digest, tag-list, prefix and stamp-function commands marked "checked"; and `bash -n` plus a dry run of the step 6 and rollback scripts on a scratch copy of the layout with stub `stop.sh` and `start.sh`
Tag scheme	`<prefix>-1.0.0-dev-0.0.N`, annotated tags. The prefix is `vasic` (part 2). Baseline: `vasic-1.0.0-dev-0.0.1` (workshop `ce62bcb`, umbrella `a55e878`)
Not covered	Release channels beyond "dev", signing, and any step that needs a credential. No credential appears or is needed in this document

Contents

1. Why revision 1 was withdrawn
2. Names, paths and remotes
3. Preconditions
4. The procedure (steps 1 to 10 and step 4a)
5. Rollback
6. Tag message template
7. What can go wrong
8. Evidence to keep for each release

1. Why revision 1 was withdrawn

Revision 1 told the operator to build with `scripts/build.sh`, hash `platform/bin/workshop-server`, run `scripts/verify.sh`, and then run `scripts/restart.sh`. Each of those steps was checked against the scripts and does not do what was claimed:

Revision 1 claim	What the script does
The hash recorded before `restart.sh` will equal `WORKSHOP_PREPARED_SERVER_SHA`	`restart.sh` always rebuilds: it calls `prepare_fresh_server` (`scripts/restart.sh:146`), which builds a **new** binary to `<bin>.new.<pid>` with a new build id and `built_at`, installs it with `commit_prepared_server` after `stop.sh`, and exports the **new** binary's hash (`restart.sh:175`). The hash can never equal the one recorded from an earlier build

Revision 1 claim	What the script does
`build.sh` is the first, safe step	A full `scripts/build.sh` runs `rm -rf "${WEB_DIR:?}"/*` (`scripts/build.sh:152`) and writes `platform/bin`. `platform/web` and `platform/bin` are the **bind-mounted directories of the live container** (`platform/compose.yml`: `./bin:/opt/workshop/bin:ro`, `./web:/opt/workshop/web:ro`). The live assets are rewritten before any verdict exists
`scripts/verify.sh` produces the verdict for the candidate	`verify.sh` probes the live stack (it resolves the live base from `platform/run*/server.json`, default `http://127.0.0.1:8087`, `scripts/verify.sh:1306`). That is the **old** deployment. It prints no candidate fingerprint
N is "the next number after the highest local tag"	`git tag -l` lists local tags only. A tag created and pushed from another clone or worktree is invisible, so the number can be reused. N must come from the remote (step 8)

The procedure below is the one whose artifacts exist for release 0.0.2.

2. Names, paths and remotes

- **Prefix.** The tag prefix is resolved by `submodules/constitution/scripts/release_prefix.sh` (11.4.151): the environment variable `HELIX_RELEASE_PREFIX`, else the same variable in the git-ignored project-root `.env`, else the lowercased snake_case project directory name. There is **`no .env.example`** in the umbrella or in `workshop` that documents the variable (checked: no `.env*` file other than `.environment-assumptions-allow`). Checked 2026-10-20: `bash submodules/constitution/scripts/release_prefix.sh` prints `vasic`. Derive the prefix with the script, never type it: `PREFIX="$(bash submodules/constitution/scripts/release_prefix.sh)"`.
- **Candidate directory.** `$HOME/.cache/workshop-release/N/` with `bin/` (the server binary), `web/` (the Angular bundle), `stamp.txt`, `bundle.hash`, `web.filehashes.txt` and, at promotion time, `pre-deploy-backup/`. It is outside every repository and outside every live mount.
- **Evidence directory.** `workshop/evidence/release/N/` (committed): `MANIFEST.json`, `verdict.json`, `BUILD_STAMP`, `SHA256SUMS`, `web.filehashes.txt`, `logs/`. **No file in it may contain a host-absolute path**: the umbrella hardcoded-path audit (`scripts/audit-hardcoded-paths.sh`) rejects any host-absolute path under the user's home directory. Write `$HOME/...` or `<scratch>/...`. Check with `grep -rnE '/hom[e]/' evidence/release/N` (must print nothing).
- **Remotes** (checked 2026-10-20 with `git remote -v`): the umbrella has `github`, `origin` and `upstream`, all `git@github.com:milos85vasic/vasic.git`; `workshop` has the same three names, all `git@github.com:milos85vasic/workshop_curriculum.git`. Each owned submodule that changes has its own remotes: read them with `git -C <submodule> remote -v` and push to every one that is a distinct URL. Fast-forward only, never forced (11.4.113).
- **Port.** The compose stack binds `${WORKSHOP_HTTP_BIND:-127.0.0.1}:${WORKSHOP_HTTP_PORT:-8087}` with `-port-fallback ${WORKSHOP_PORT_FALLBACK:-20}`, so if 8087 is taken the server binds a nearby port. Read the real address from `platform/run*/server.json` (the file `verify.`

`sh` and `status.sh` read) rather than assuming 8087. Below, `$LIVE_URL` means that address.

3. Preconditions

1. **The increment is merged on `main`** and `git fetch --all` shows nothing newer that belongs in it. Integrate by merging onto the latest `main`; never rebase published history.
2. **Independent review is GO** for every change in the release (11.4.142).
3. **The release is built from a clean worktree** of the commit to release: `git worktree add <worktree-parent>/rel-N <commit>`; `git -C <worktree> status --porcelain` is empty. A dirty tree is not a release (the binary would stamp `source_dirty=true`).
4. **Heavy commands are admitted**, with the umbrella script by its absolute path, because `agent-admit.sh` lives in the umbrella's `scripts/` and `run-isolated.sh` in `workshop`'s `scripts/`; no single directory contains both:

   ```
   UMBRELLA=<path of the umbrella checkout>
   cd <workshop checkout or release worktree>
   bash "$UMBRELLA/scripts/agent-admit.sh" --slots 4 --tag rel --wait 1800 -m 6G -T
           3600 -- <command>
   ```

   A refusal means wait and retry; never bypass it.
5. **Nothing uses a model on recordings, transcripts or textbooks** (token rule).
6. **A hardlinked backup of the live data** exists before anything is replaced (Constitution §9): the registry (`curriculum/passages.jsonl`), `platform/prebuilt` and the index volume. Step 5 adds the binary, stamp and `index.html` backups specific to this procedure.

4. The procedure

Every script is three-valued: 0 good, 1 a real problem, 2 could not determine. **A 2 is never a pass.** Steps 1 to 5 (including 4a) change nothing that is served (step 5 only copies files out). Step 6 is the only step that changes the live stack; step 7 reads it; steps 8 to 10 touch the repositories.

Step 1. Build once, into the candidate directory

From the clean release worktree, never from the live checkout and never with `scripts/build.sh`:

```
N=<next number, see step 8>             # decided now, re-checked at step 8
CAND="$HOME/.cache/workshop-release/$N"; mkdir -p "$CAND/bin"
COMMIT="$(git rev-parse HEAD)"; SHORT="$(git rev-parse --short HEAD)"
NOW="$(date -u +%Y-%m-%dT%H:%M:%SZ)"; BID="$SHORT-$(date -u +%Y%m%dT%H%M%SZ)"
```

```
( cd platform/backend && CGO_ENABLED=0 go build -trimpath \
    -ldflags "-X main.buildID=$BID -X main.sourceCommit=$COMMIT -X
        main.sourceDirty=false -X main.builtAt=$NOW" \
    -o "$CAND/bin/workshop-server" ./cmd/workshop-server )
( cd platform/frontend && npm ci && npx ng build --configuration production )   # then
        copy the dist
                                                                                #
        browser dir to "$CAND/web"
```

The stamp flags are the ones `scripts/_common.sh` `build_server_stage` uses (line 684). For 0.0.2 the logs show `BUILD_RC=0 VET_RC=0` and `NGBUILD_RC=0` (`evidence/release/0.0.2/logs/go-build-vet.log`, `ng-build.log`) and the toolchain is in `MANIFEST.json`. The exact build command lines of 0.0.2 are **UNCONFIRMED** (the logs keep the results, not the commands). Heavy: admit both builds.

Also write `$CAND/bin/workshop-server.stamp`. What a correct one contains, and why releases 0.0.3 and 0.0.4 installed an incomplete hand-written one, is in step 6 ("What a correct stamp looks like"). The repository's own writer is `build_server_stage` in `scripts/_common.sh`; sourcing it from the release worktree and calling `build_workshop_server_stage_only` would stage a binary and a correct stamp under that worktree's `platform/bin`, which can then be copied into `$CAND`. That route is **UNCONFIRMED** (it was not run; only `_server_inputs_state` was called, read-only).

Built once means: the files in `$CAND` are the only artifact. Nothing after this step rebuilds.

Step 2. Fingerprint the candidate

```
sha256sum "$CAND/bin/workshop-server"            # binary_sha256
( cd "$CAND/web" && find . -type f ! -name BUILD_STAMP | sort | xargs -d '\n'
        sha256sum ) \
    > "$CAND/web.filehashes.txt"
sha256sum "$CAND/web.filehashes.txt"             # bundle_hash
```

Checked 2026-10-20 against the 0.0.2 candidate directory: this reproduces its recorded `bundle_hash` (`2e9831ed...`, 181 files) exactly, so the "bundle hash" of 0.0.2 is **the sha256 of the sorted `sha256sum` listing of every bundle file except `BUILD_STAMP`**. Two cautions from that check: the `sort` is the shell's locale sort (with `LC_ALL=C` the digest differs: `5a746988...`), so fix the locale and record it; and the hash is of the listing, not a tree hash. The bundle also carries a small `web/BUILD_STAMP` (key=value lines: `frontend_source_git_sha`, `repo_commit`, `built_at`, `bundle_hash`; it is excluded from the hash it records, so write it after the hash is known). Its producer in the repository, `scripts/write-build-info.sh`, exists only on the unmerged branch `s12-019-promotion`: by hand until that merges (**UNCONFIRMED** how 0.0.2 wrote it). Write the two digests to `$CAND/fingerprint.txt` as `binary_sha256=<64 hex>` and `bundle_hash=<64 hex>` (step 6 and the rollback read the first line from it). Record `{commit, tag_commit (filled in at step 4), binary_sha256, bundle_hash}` in `MANIFEST.json` and in `verdict.json` as `candidate_fingerprint`. Also record the one-line summary `stamp.txt` (commit, dirty flag, `built_at`, build id) that 0.0.2 kept.

Step 3. Verdict against that fingerprint, without touching the live stack

`verify.sh` cannot give this verdict (part 1). Run the suites and gates individually, as 0.0.2 did (`evidence/release/0.0.2/logs/run-gates.sh` is the driver; `logs/gates.tsv` is its result table) and write `verdict.json` with one row per check: `name`, `expected`, `observed`, `rc`, `status`, `reason`.

- **Unit and component suites** against the release worktree: `go build ./... && go vet ./...`, `go test`, the Angular unit tests (0.0.2 ran them in fixed order and with random seed 11111), the pipeline tests, and the gates under `platform/gates/` and `scripts/` that need no live server.

- **A server for the checks that need one** is the isolated server, **never `:8087`**, and it must run the candidate binary, not a fresh build:

  ```
  WORKSHOP_ISOLATED_BUILD="$CAND/bin/workshop-server" \
    bash "$UMBRELLA/scripts/agent-admit.sh" --slots 4 --tag rel --wait 1800 -m 6G -
          T 3600 -- \
      bash scripts/run-isolated.sh --exec -- bash -c 'curl -s "$ISOLATED_URL/api/
          health"'
  ```

  `WORKSHOP_ISOLATED_BUILD` makes `run-isolated.sh` copy the prebuilt binary instead of running `go build` (`scripts/run-isolated.sh` header, "ENVIRONMENT"). Use the `--exec` form under `agent-admit`: a detached start dies with the admitted command. The isolated server serves six synthetic registry rows and no web bundle, so it proves the **binary's** behaviour on synthetic data. It does not prove the bundle (that is the Angular suite plus the bundle hash) and does not prove the real corpus.

- **Record every non-pass honestly, in this vocabulary.** Per check (`status`): `PASS`; `FAIL`; `UNDETERMINED` (the check could not run: an environment fault or a missing instrument; never a pass); `FLAKY` (the check passed and failed on reruns of the same inputs: 0.0.4 recorded `gate:ingest-textbooks-test` as `FLAKY` after 1 failure in 5 runs; a `FLAKY` row is neither a pass nor a waived failure, and its root cause is stated or is stated `UNCONFIRMED`). Overall (`overall`), as actually used: `FAIL` (0.0.2: three baseline-identical failures, waived afterwards in `docs/releases/0.0.2.md`); `BLOCKED` (0.0.3: a `FAIL` that is new and unwaived, with a `blocking_assessment` saying what it touches); `RELEASABLE_WITH_WAIVERS...` (0.0.4: every non-pass is a baseline-identical failure with a numbered waiver `W1`..`Wn`, an environment `UNDETERMINED` with its reason, or a named open decision; 0.0.4 spelled the open decision into the value: `RELEASABLE_WITH_WAIVERS_PENDING_ORCHESTRATOR_DECISION_ON_ingest-textbooks-test_FLAKE`); and `PASS` (every row passes; none of the three releases reached it). Use these words; do not invent a fourth one without defining it in `verdict.json`.

- **A waiver** is a row with an id, what it covers, the evidence that the failure is identical at the previous release (or is outside everything the release deploys) and who granted it (operator or delegate, below). 0.0.2 waived three baseline-identical

failures this way and listed 5 undetermined checks; 0.0.4 carried those same three as `W1` to `W3`.

- **Shipping over a `BLOCKED` or `FAIL` verdict is allowed only through an explicit, recorded waiver.** It must name each blocking finding, say why it does not touch the deployed artifacts (the server binary and the web bundle) and how it will be fixed, and say who granted it: the **operator**, or the **delegate** (the orchestrating agent acting under the operator's standing approval). A delegate waiver is **not** an operator decision and is recorded as such (it is amendable by the operator, and it is listed in the operator-decisions log of that date). Release 0.0.3 is the precedent: its `verdict.json` says `BLOCKED` (two new findings, N1 a test-isolation defect and N2 six exempt registry rows, both in non-server files), it was tagged and deployed on a delegate waiver on 2026-10-21, and the two findings were fixed in 0.0.4 (the log entry is `OPERATOR-DECISIONS-2026-10-21.md`, W-01). The verdict file itself is **not edited** to say otherwise: a later reader must be able to see that the tool said `BLOCKED`.
- The full-suite retest of 11.4.40 is required for a non-dev tag; for a `dev` tag the tag message says whether it was run.

If any check is `FAIL` without a recorded waiver, stop. The candidate is releasable only with a verdict tied to the step 2 fingerprint (11.4.236).

Step 4. Prepare the evidence

Copy `stamp.txt`, `web.filehashes.txt`, the logs and the two JSON files into `workshop/evidence/release/N/`, write `SHA256SUMS`, then run `grep -rnE '/hom[e]/' evidence/release/N` (must be empty). Write `docs/releases/N.md` (what changed, known limitations, validation status). Commit in the release worktree with explicit paths. **Two commits are recorded, and they may differ.** The **build commit** is the one the artifact was compiled from (`MANIFEST.json` `commit`; it is what `/api/health` reports as `source_commit`). The **tag commit** is the commit the tag points at (`MANIFEST.json` `tag_commit`), usually the later evidence-and-notes commit. The claim of revision 2, that the tag commit differs from the build commit by documentation and evidence only, was **not true for every release**:

Release	Build commit	Tag commit	`git diff --stat <build> <tag>` outside `evidence/release/` and `docs/` (checked 2026-10-21)
0.0.2	`e16a196`	`30e61ad`	none (evidence and notes only)
0.0.3	`d6b988e`	`1cf59de`	none
0.0.4	`cbe4628`	`675ca48`	3 files: `platform/backend/cmd/workshop-server/wire_hooks_test.go` (a test), `platform/gates/.cross-registry-attribution-allow` (a gate allow-list), `scripts/prove-wire-hooks.sh` (a gate)

The rule: the difference between the build commit and the tag commit **must be limited to tests, gate allow-lists and gate scripts, documentation and evidence**, nothing under `platform/frontend` and no non-test file under `platform/backend`; the difference is listed in the release notes with the output of `git diff --stat <build> <tag>` and the manifest says it in words (0.0.4's `MANIFEST.json` `note` does). If the difference touches the deployed artifacts, the candidate is not the tag: build again.

Step 4a. Pre-deploy safety boot on a copy of the live store

Step 3's isolated server serves six synthetic rows and no bundle, so it cannot say whether the candidate **boots on the real data and the real accounts**. Release 0.0.3 added this check (its "H3" item) and 0.0.4 ran it (`evidence/release/0.0.4/logs/predeploy-safety.txt`). It changes nothing that is served.

1. **Copy, never mount.** With plain `cp -a` (no hardlinks: a hardlink shares the inode with the live file) copy into a scratch directory outside every repository, mode `0700`: the live **auth store**, the live **index** directory and the live **curriculum** (registry, taxonomy, the textbook catalogue). Take the host paths from the bind mounts and `-state`/`-index`/`-curriculum`/`-registry` flags in `platform/compose.yml`. Copy **no** `.env`, no access-token file, no key file. The auth store holds password hashes: it is credential-bearing, so it stays in the scratch directory, is never printed, never committed and is deleted at the end.
2. **Boot the candidate, not a fresh build, on a scratch port.** Use the candidate binary from `$CAND/bin`, `-http 127.0.0.1:<port from 18200 upward> -port-fallback 0`, the scratch copies for every data flag, `-web "$CAND/web"`, `-answer-provider none` and **`no -ollama`** (no embedding or model contact), under `env -i` with only `PATH` and `HOME`, so the process sees **`no WORKSHOP_ENV` and `no WORKSHOP_DEV_MODE`**, exactly as the live container does. **`Never :8087`**, never a process you did not start. Run it under `agent-admit` like any heavy command. The exact command line used for 0.0.4 was not kept (the log keeps its header and results): **UNCONFIRMED** as a replay.
3. **Probe.** `GET /api/health` is 200 with `status: ok` and the candidate's `build`; `GET /textbooks`, `/profile` and `/` are 200 (the SPA shell); the data routes answer **401** without a session (`/api/textbooks`, `/api/textbooks/citations`, `/api/profile`). For the authenticated half, set a **throwaway password on the COPY's account only** (with the repository's password tool, pointed at the copy), log in against the scratch server, and require 200 from `/api/textbooks`, `/api/profile`, `/api/profile/reports` and `/api/search`. Count 5xx: it must be 0. Never print the token or the password.
4. **Boot a second time in the worst case, `WORKSHOP_ENV=production`,** on the same copy. This is **not** the live configuration; it answers whether arming the production-mode guard would refuse to start. Release 0.0.3 measured exactly that (`evidence/release/0.0.3/logs/h3-boot-safety.txt`): under the live configuration the candidate boots and `/api/health` is 200; under `WORKSHOP_ENV=production` it **refuses to start** because the `milosvasic` account still carries its tracked example password hash (the `rami` account does not). A refusal here is the guard working as designed, not a candidate defect: 0.0.3 recorded both boots as `PASS` rows and

added an **operator action** (rotate the account's password with the repository's password tool, before ever arming `WORKSHOP_ENV=production`); 0.0.4 did not re-run the production-mode boot and carried the action over as an `UNDETERMINED` row. Until the rotation is done the guard cannot be armed. The password rotation is deferred by the operator (`OPERATOR-DECISIONS-2026-10-21.md`, D-02). Also read the first boot's own lines (the textbook state line, the learning-catalog findings, the answering state): they are evidence.

5. **Stop your own pid, delete the scratch copies,** write the probe table to `logs/predeploy-safety.txt` (host paths replaced by `$HOME`; checked with the same `grep`), and add a `pre-deploy: ...` row to `verdict.json`.

Step 5. Back up what will be replaced

```
BK="$CAND/pre-deploy-backup"; mkdir -p "$BK"
cp -p platform/bin/workshop-server       "$BK/workshop-server"
cp -p platform/bin/workshop-server.stamp "$BK/workshop-server.stamp"
        # the multi-line stamp
cp -p platform/web/index.html            "$BK/index.html"
[ -f platform/web/BUILD_STAMP ] && cp -p platform/web/BUILD_STAMP "$BK/BUILD_STAMP"
echo "binary_sha256=$(sha256sum "$BK/workshop-server" | cut -d' ' -f1)" > "$BK/
        backup.sha"
```

The first three files are what 0.0.2 kept in `pre-deploy-backup/`; `BUILD_STAMP` and `backup.sha` are added in revision 3 so that the rollback restores the whole page state and can prove it restored the backed-up bytes. Run it from the **live** checkout. Also make the part 3 item 6 data backup if the release changes data.

Step 6. Promote without rebuilding

Do not call `build.sh` or `restart.sh`. Do not `rm -rf` anything under `platform/web`. The whole step is **one script run**, so that there is no gap between its parts in which the stack serves a mixture, and it uses **absolute paths only** (revision 2 used `cd "$CAND/web"` followed by relative `scripts/...` commands, which fails when pasted: the working directory is no longer the live checkout).

Order, and why:

1. **Hashed web assets first, additively.** The Angular build emits content-hashed files, so copying the new ones in changes nothing the running (old) `index.html` references. Everything except `index.html` and `BUILD_STAMP` is copied, each by a temporary name and `mv` (atomic on one filesystem).
2. **Stop the stack** (`stop.sh`: 0 stopped; on 1 or 2 do not continue).
3. **Install the binary and its stamp** by temporary name and `mv`, and prove the installed bytes equal the candidate's recorded `binary_sha256`.
4. **Swap `index.html` and `BUILD_STAMP` while the server is down.** Doing this after the stop (and not before it, as revision 2 did) removes the window in which the **new** page was served by the **old** binary: with the page swapped first, a learner who loaded `/` between that swap and the restart got a page whose API the old binary did not have.

5. **Start with the candidate's hash handed over for that one command only.** `start.sh`'s `ensure_fresh_server` (`scripts/_common.sh:836`) compares the installed binary with `WORKSHOP_PREPARED_SERVER_SHA`, logs "prepared and installed ... (sha256 confirmed); not rebuilding" and refuses (exit 2) on a mismatch. The value must be the **candidate's recorded `binary_sha256`** (step 2), **not** the hash of the file just installed: computing it from the installed file makes the comparison true by construction, so a wrong file installed by mistake would be confirmed. The script puts the variable on the `start.sh` command line only, never `exported` (see the hazard below).

```
#!/usr/bin/env bash
# promote.sh N LIVE  (both arguments required; LIVE is the absolute path of the live
        workshop checkout)
set -euo pipefail
N="${1:?release number}"; LIVE="${2:?absolute path of the live workshop checkout}"
case "$LIVE" in /*) ;; *) echo "LIVE must be an absolute path" >&2; exit 2 ;; esac
CAND="$HOME/.cache/workshop-release/$N"; BK="$CAND/pre-deploy-backup"
WEB="$LIVE/platform/web"; BIN="$LIVE/platform/bin"
EXPECT="$(sed -n 's/^binary_sha256=//p' "$CAND/fingerprint.txt")"
        # written in step 2
[ -n "$EXPECT" ] || { echo "no binary_sha256 in $CAND/fingerprint.txt" >&2; exit 2; }
[ "$(sha256sum "$CAND/bin/workshop-server" | cut -d' ' -f1)" = "$EXPECT" ] \
    || { echo "the candidate binary does not match its recorded fingerprint" >&2; exit
        2; }
[ -f "$BK/workshop-server" ] && [ -f "$BK/index.html" ] \
    || { echo "the step 5 backup is missing: nothing was changed" >&2; exit 2; }
trap 'echo "promote.sh FAILED at line $LINENO: the stack may be down; follow part 5
        (rollback)" >&2' ERR

# 6.1 hashed web assets, additive (the old index.html does not reference them)
( cd "$CAND/web" && find . -type f ! -name index.html ! -name BUILD_STAMP -print0 ) \
  | while IFS= read -r -d '' f; do
      mkdir -p "$WEB/$(dirname "$f")"
      cp -p "$CAND/web/$f" "$WEB/$f.new.$$" && mv -f "$WEB/$f.new.$$" "$WEB/$f"
    done

# 6.2 stop (exit 0 only; set -e ends the script on 1 or 2)
( cd "$LIVE" && bash "$LIVE/scripts/stop.sh" )

# 6.3 binary and multi-line stamp, then prove the bytes
cp -p "$CAND/bin/workshop-server" "$BIN/workshop-server.new.$$" && mv -f "$BIN/
        workshop-server.new.$$" "$BIN/workshop-server"
if [ -f "$CAND/bin/workshop-server.stamp" ]; then
  cp -p "$CAND/bin/workshop-server.stamp" "$BIN/workshop-server.stamp.new.$$" \
    && mv -f "$BIN/workshop-server.stamp.new.$$" "$BIN/workshop-server.stamp"
fi
[ "$(sha256sum "$BIN/workshop-server" | cut -d' ' -f1)" = "$EXPECT" ] \
    || { echo "the installed binary differs from the candidate: STOP, roll back" >&2; e
        xit 2; }

# 6.4 the page and its stamp, last, while the server is down
cp -p "$CAND/web/index.html" "$WEB/index.html.new.$$" && mv -f "$WEB/index.html.new.$
        $" "$WEB/index.html"
cp -p "$CAND/web/BUILD_STAMP" "$WEB/BUILD_STAMP.new.$$" && mv -f "$WEB/BUILD_STAMP.new.
        $$" "$WEB/BUILD_STAMP"

# 6.5 start, confirming the CANDIDATE's hash (variable on this one command only)
( cd "$LIVE" && WORKSHOP_PREPARED_SERVER_SHA="$EXPECT" bash "$LIVE/scripts/start.sh" )
trap - ERR
echo "promoted release $N; now do step 7"
```

(LIVE is the live checkout; old hashed files are left in place, and pruning them is a separate, later, operator action. The script was syntax-checked with bash -n and **dry-run on a scratch copy of the layout** with stub stop.sh and start.sh (a scratch directory standing in for $LIVE, no stack touched): the dry run is recorded in part 4, "Checked in revision 3" below. It was never run against a real stack by its author: **UNCONFIRMED** on a real promotion.)

If start.sh returns 1 or 2, go to part 5. Do not start by hand.

Hazard: a later plain start.sh or restart.sh replaces the released binary (it breaks 11.4.264). WORKSHOP_PREPARED_SERVER_SHA only protects the one start that carries it. Any *later* start.sh or restart.sh without it, on a host with a Go toolchain, goes through prepare_fresh_server (_common.sh): it **rebuilds the server from the live checkout** and installs the result over the released binary, with a new build id and the live checkout's HEAD as source_commit. The served binary then is not the candidate, and nothing in the stack's own output says so unless you read /api/health. After any restart of the live stack, repeat step 7 and compare the installed binary's sha256 with the recorded one. And start.sh itself (scripts/start.sh:68-70) runs scripts/build.sh if the control-plane binary (workshop-boot) is missing or the server binary is not executable; build.sh runs rm -rf "${WEB_DIR:?}"/* and rebuilds the bundle, so it empties the promoted platform/web. Both are the reason the release notes must say, in one line, how the stack may be restarted after the release (with the candidate's hash, or by promoting the next release).

What a correct stamp looks like. The binary's own identity (/api/health: build, source_commit, built_at) comes from the linker flags of step 1 and does not depend on the stamp. The file platform/bin/workshop-server.stamp is separate: build_server_stage (scripts/_common.sh, the printf block that writes the stamp) writes eight key=value lines, then the module lines of _server_inputs_state: commit (full sha of the build commit), dirty, src_dirty, allow_dirty, bin_sha256 (of the binary), build (the build id), built_at, source (<module dir relative to the checkout>/<package>, e.g. platform/backend/cmd/workshop-server), and one module=<absolute dir> head=<sha> dirty=<n> line per input module (the server module plus the owned submodules it compiles: checked on 2026-10-21, _server_inputs_state prints eight such lines for the d1-docs worktree). Releases 0.0.3 and 0.0.4 installed a **hand-written** stamp with the **eight keys and no module= lines**, and a free-text source = (evidence/release/0.0.4/server.stamp reads source=release-0.0.4 (built once from cbe4628, hash-handoff install)). That stamp is consulted only when **no Go toolchain** is on PATH and no hash is handed over (_common.sh:783-813). Read against that code it would be refused there, for two independent reasons: its commit is the build commit, not the live checkout's HEAD ("built from X but HEAD is Y"), and it has no module= lines for the check grep -qxF "module=$mod head=$mhead dirty=0" to find. (That refusal path was **read, not executed**: UNCONFIRMED.) Two consequences. First, a no-Go host can run a candidate built elsewhere only through the hash hand-over of step 6.5 (or WORKSHOP_ALLOW_STALE_BINARY=1, an operator decision), because the stamp check compares against the live checkout, which is never the release worktree. Second, the module= lines carry **host-**

absolute directories: a stamp is a host file, not evidence. The copy in `evidence/release/N/` must have those paths replaced (`$HOME/...`) like every other evidence file (part 2), and a stamp that is meant to pass the no-Go check must be produced by `build_server_stage` on the host that will run it, not typed.

Step 7. Confirm what is served

```
curl -s "$LIVE_URL/api/health"
```

`build` must equal the candidate build id (0.0.2: `e16a196-20261020T165046Z`), `source_commit` the **build commit** (not the tag commit), `source_dirty` `false`, `built_at` the stamp's time. Then compare the served page with the bundle:

```
curl -s "$LIVE_URL/" | sha256sum        # equals:
sha256sum "$CAND/web/index.html"
```

(`index.html` is served unmodified from `./web`: that equality is the check; if the server rewrites it the comparison is **UNCONFIRMED** and the fallback is to compare the hashed chunk file names that the served page references with `web.filehashes.txt`.) Run `bash scripts/status.sh` (RUNNING / STOPPED / DEGRADED / UNKNOWN) and then `bash scripts/verify.sh` **as a smoke check of the new live stack**; it is not the release verdict. Redo `grep -rnE '/hom[e]/'` on any new evidence.

Step 8. Tag

Decide N from the **remotes**, not from local tags (part 1). In each repository to be tagged:

```
git fetch --tags origin
git ls-remote --tags origin "$PREFIX-1.0.0-dev-0.0.*"      # highest N on the remote,
        after the ^{} lines
git tag -l "$PREFIX-1.0.0-dev-0.0.*" | sort -V | tail -1   # local, for comparison only
```

N is one more than the highest number seen on **any** remote of **any** of the three repository kinds. If the number you built under in step 1 is no longer free, the candidate directory and the evidence are named for a number that will not be used: rename them before tagging, or build again. Never move or delete a published tag; a wrong tag is superseded by the next number.

Tag three kinds of repository, each at the commit the release actually uses:

1. **workshop**, at the **tag commit** of step 4 (0.0.3: `1cf59de`, 0.0.4: `675ca48`);
2. the **umbrella**, at the commit whose gitlink pins that workshop commit (0.0.3: `4c7a1d2`, 0.0.4: `de2761b`);
3. every **owned submodule that changed** since the previous release tag.

```
git -C workshop tag -a "$PREFIX-1.0.0-dev-0.0.$N" -m "<message, part 6>" <workshop-
        commit>
git tag -a "$PREFIX-1.0.0-dev-0.0.$N" -m "<message, part 6> (umbrella commit pinning
        workshop <short>)" <umbrella-commit>
```

A gitlink bump and the matching `helix-deps.yaml` `ref` go in **one** change, otherwise cascade check C9 reports DRIFT. Bumping a submodule other than `submodules/`

`constitution` is an operator decision. Note that an annotated tag has its own object id: the commit it points to is `git rev-parse "<tag>^{commit}"` (for 0.0.2 the umbrella tag resolves to the umbrella commit `368ab84`, the workshop tag to `30e61ad`).

Step 9. Push, fast-forward only, through the pre-push gates

The pre-push hook runs the umbrella gates and can take long enough for an idle SSH connection to be dropped. Keep the connection alive:

```
export GIT_SSH_COMMAND='ssh -o ServerAliveInterval=30 -o ServerAliveCountMax=40'
git fetch --all                                  # integrate first: merge onto the
        latest main
git push origin main                             # fast-forward only; git refuses
        anything else
git push origin "refs/tags/$PREFIX-1.0.0-dev-0.0.$N"
```

Repeat for **every distinct remote** of each repository (part 2; `github`, `origin` and `upstream` currently name one URL, so one push per repository reaches it, but read `git remote -v` rather than trusting this sentence), and for each tagged submodule. **Never** `--force`, `--force-with-lease`, a `+ref`, `--no-verify`, or a history rewrite (11.4.113). A rejected non-fast-forward push means: fetch, merge onto the latest main, push again.

Step 10. Verify the publication

```
git ls-remote origin refs/heads/main "refs/tags/$PREFIX-1.0.0-dev-0.0.$N" "refs/tags/$P
        REFIX-1.0.0-dev-0.0.$N^{}"
bash scripts/verify-manifest-pins.sh             # umbrella: helix-deps refs equal
        gitlinks
bash scripts/verify-submodule-remote-sync.sh     # umbrella: gitlinks vs remotes
```

Each remote's `main` equals the local `main`, the tag object matches, and the peeled `^{}` line is the commit you meant.

Checked in revision 3

Run by the author on 2026-10-21 (evidence on the `workshop` branch `d1-docs`, under `evidence/d1/`: `promote-dryrun.txt`, `run-isolated-evidence.txt`, `stamp-state-check.txt`). All against scratch directories; the live stack was not touched and nothing used `:8087`.

- **Tag commit versus build commit** (step 4 table): `git diff --stat <build> <tag>` outside `evidence/release/` and `docs/` for 0.0.3 (empty) and 0.0.4 (three files).
- **The stamp writer's module lines** (step 6): `_server_inputs_state` was called read-only from the `d1-docs` worktree and printed eight `module=...` lines (host-absolute directories; they are not copied into the evidence file).
- **Step 6 and the rollback script**: extracted from this document, `bash -n` clean, and run against a scratch layout (`platform/bin`, `platform/web`, stub `scripts/stop.sh` and `scripts/start.sh` that print their working directory and the value of `WORKSHOP_PREPARED_SERVER_SHA`). Results: a normal promotion exits 0 and

the stub `start.sh` receives the candidate's hash; a wrong fingerprint exits 2 with nothing changed; a relative `LIVE` exits 2; a failing `stop.sh` exits 2 and prints the rollback pointer with the old binary still in place; the variable is not left set in the caller's shell; a rollback restores binary, page and `BUILD_STAMP` and hands `start.sh` the backup's hash; a tampered backup is refused with nothing changed. **No real stack, no real `start.sh`.**

- **The isolated server with a prebuilt release binary** (step 3): a fresh raw run with the 0.0.4 binary was attempted on 2026-10-21 and **refused by `agent-admit`** on every attempt (the app.slice working set stood at 87 to 89 percent against a threshold of 85); admission was not bypassed. The `--exec` form under `agent-admit` is therefore backed only by the raw run 2 in `run-isolated-evidence.txt` (built from the tree); the prebuilt-binary variant is **UNCONFIRMED** by captured output (run 3 of revision 2 was a paraphrase and is relabelled as such).

5. Rollback

Rollback uses the backup directory from step 5, the same no-rebuild rules, and the same shape as step 6: one script, absolute paths, the hash handed to `start.sh` for that one command only, and the hash being the **backup's own** (`sha256sum "$BK/workshop-server"`, recorded in step 5), not the hash of whatever is installed.

```
#!/usr/bin/env bash
# rollback.sh N LIVE   (the step 5 backup of release N must exist)
set -euo pipefail
N="${1:?release number}"; LIVE="${2:?absolute path of the live workshop checkout}"
case "$LIVE" in /*) ;; *) echo "LIVE must be an absolute path" >&2; exit 2 ;; esac
BK="$HOME/.cache/workshop-release/$N/pre-deploy-backup"; WEB="$LIVE/platform/web";
        BIN="$LIVE/platform/bin"
WANT="$(sed -n 's/^binary_sha256=//p' "$BK/backup.sha")"
[ -n "$WANT" ] && [ "$(sha256sum "$BK/workshop-server" | cut -d' ' -f1)" = "$WANT" ] \
    || { echo "the backup binary does not match backup.sha (step 5): nothing was
        changed" >&2; exit 2; }
( cd "$LIVE" && bash "$LIVE/scripts/stop.sh" )
cp -p "$BK/workshop-server" "$BIN/workshop-server.new.$$" && mv -f "$BIN/workshop-
        server.new.$$" "$BIN/workshop-server"
cp -p "$BK/workshop-server.stamp" "$BIN/workshop-server.stamp.new.$$" && mv -f "$BIN/
        workshop-server.stamp.new.$$" "$BIN/workshop-server.stamp"
cp -p "$BK/index.html" "$WEB/index.html.new.$$" && mv -f "$WEB/index.html.new.$$"
        "$WEB/index.html"
if [ -f "$BK/BUILD_STAMP" ]; then
  cp -p "$BK/BUILD_STAMP" "$WEB/BUILD_STAMP.new.$$" && mv -f "$WEB/BUILD_STAMP.new.$$"
        "$WEB/BUILD_STAMP"
fi
[ "$(sha256sum "$BIN/workshop-server" | cut -d' ' -f1)" = "$WANT" ] || { echo
        "restored binary differs from the backup" >&2; exit 2; }
( cd "$LIVE" && WORKSHOP_PREPARED_SERVER_SHA="$WANT" bash "$LIVE/scripts/start.sh" )
echo "rolled back to the step 5 backup of release $N; now repeat step 7 and expect the
        PREVIOUS build id"
```

The new hashed files may stay: the restored `index.html` does not reference them. Do not delete the failed release's tag if it was pushed; supersede it with the next number. The script was `bash -n` checked and dry-run on the same scratch layout as `promote.sh` (part 4, "Checked in revision 3"); against a real stack the rollback path has **never been exercised** for any release (UNCONFIRMED).

6. Tag message template

```
<one line: what this release adds>.
Build: workshop <short sha>, artifact sha256 <first 12>, verdict <PASS/waived/..>
(verdict.json kept at evidence/release/N/).
Full-suite retest (11.4.40): <run | NOT run>. <If not run, say so; dev markers may omit
it, they may not hide it.>
(umbrella commit pinning workshop <short>)
```

7. What can go wrong

Symptom	Cause	Action
`start.sh` says the sha256 “no longer equals WORKSHOP_PREPARED_SERVER_SHA”	the binary on disk changed between install and start, or the variable was computed from another file	recompute from the installed file; if it differs from the candidate’s `binary_sha256`, the install did not copy the candidate: restore and retry
`start.sh` builds instead of confirming	`WORKSHOP_PREPARED_SERVER_SHA` was not on the `start.sh` command line (for example the step 6 script was split up, or `start.sh` was run alone)	a rebuilt binary is not the candidate: roll back (part 5) and redo step 6 as the one script; do not `export` the variable into an interactive shell (step 6 hazard)
`/api/health` shows `source_dirty: true` or another build id	the served binary is not the candidate	not a release: roll back
Served `index.html` differs from the bundle’s	`index.html` was copied before its assets, or an old file was served from cache	re-copy; step 7
`verify.sh` exit 2	an instrument could not run	not a pass; record the gap
Push rejected (non-fast-forward)	upstream moved	fetch, merge onto the latest main, push again; never force
Pre-push hook output stops and the push fails with a broken pipe	the SSH connection idled out during the gates	`GIT_SSH_COMMAND` with `ServerAliveInterval` (step 9)
Tag already exists on a remote	N taken by another clone	step 8: take the next number; never move a tag
C9 reports DRIFT	gitlink and `helix-deps.yaml` ref moved separately	stage them together
A restart after the release serves a different binary	a later plain `start.sh` or `restart.sh` without `WORKSHOP_PREPARED_SERVER_SHA`, on a host with Go, rebuilt from the live checkout and replaced the released binary	step 6 hazard: re-run step 7, compare the sha256 with `fingerprint.txt`; restart only with the candidate’s hash, or promote the next release
`platform/web` is suddenly empty or rebuilt	`start.sh` ran `build.sh` because `workshop-boot` was missing or the server binary was not executable (`scripts/start.sh:68-70`), and `build.sh` does `rm -rf "${WEB_DIR:?}"/*`	roll back (part 5); build the missing control-plane binary deliberately, not as a side effect of a start
`start.sh` refuses on a host without Go: “built from X but HEAD is Y”, or “built from a different revision of” a module	the stamp’s `commit` is the build commit, or it has no `module=` lines (a hand-written stamp), and the check compares against the live checkout	use the hash hand-over of step 6.5; produce stamps with `build_server_stage` (step 6, “What a correct stamp looks like”)
The pre-deploy safety boot refuses	the production-mode guard met the live auth store’s example account hash	not a candidate defect (the guard working): listed as an operator action (rotate with the repository’s password tool); record it as a `PASS` of the guard plus the action, or `UNDETERMINED` if the production-mode boot was not re-run
`audit-hardcoded-paths` rejects the evidence	a log contains a host-absolute home-directory path	replace with `$HOME` or `<scratch>`, refresh `SHA256SUMS`

8. Evidence to keep for each release

The commit hashes (workshop build commit **and** tag commit, umbrella, each tagged submodule) and the `git diff --stat` between the two workshop commits, `binary_sha256` and `bundle_hash` with the method that produced the latter (`fingerprint.txt`), the stamp (host paths replaced), the pre-deploy safety log of step 4a, `verdict.json` with its waivers, the backup path (`$HOME/.cache/workshop-release/N/pre-deploy-backup`), the `/api/health` output after start and the served-page comparison, the `git ls-remote` output per remote, and the tag messages. A release without these is a claim, not a record.